

flvs.bran — System Blueprint

flvs.bran — System Blueprint

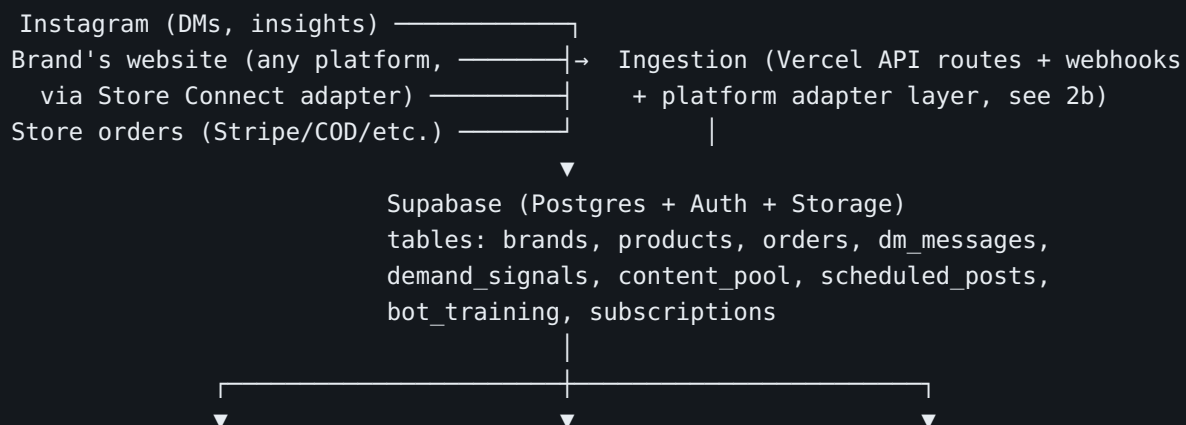
AI automation platform for clothing brand owners, sold as a subscription. Built to run on the same stack as flvs.life (Supabase + Vercel), so your brand becomes customer #1 and the proof of concept. The core engines (restock intelligence, DM chatbot, post scheduler, fulfillment automation) are not apparel-specific — section 9 covers other online business types this can extend to once the clothing vertical is proven.

1. What it is

A web dashboard brand owners subscribe to. It connects to their Instagram, their website/store, and their supplier info, then runs four engines:

1. **Store Connect** — plugs into whatever website/platform the brand owner already has (Shopify, WooCommerce, Squarespace, Wix, custom, or none) so stock and order data flows in without rebuilding their site.
 2. **Restock AI** — decides what to reorder and what to hold.
 3. **Order Fulfillment** — reads incoming site orders and books the courier automatically (oDeliver in Trinidad), from pickup at your point of sale to delivered.
 4. **DM Chatbot** — answers customers 24/7 in the brand's own voice and dialect.
 5. **AI Post Scheduler** — picks content, day, and time; owner just confirms.
 6. **Back-end automations** — cart recovery, hype detection, reports, win-backs.
-

2. Architecture (Supabase + Vercel)



Vercel Cron jobs
(nightly demand scan,
weekly report, post
auto-publish)

Claude API calls
(chatbot replies,
reorder reasoning,
post captioning)

Next.js dashboard
(what the brand
owner logs into)

Core pieces and why:

- **Next.js on Vercel** — dashboard + API routes + webhook receivers. You're already deploying here.
- **Supabase** — Postgres for all data, Auth for brand-owner logins, Storage for content pool images/videos, Row Level Security so each brand only sees its own data (critical for a multi-tenant subscription product).
- **Claude API** — powers the chatbot, reorder explanations, and caption writing.
- **Meta Graph API** — Instagram DMs (Messenger Platform for IG), publishing (Content Publishing API), and insights (best times, top formats). Requires a Meta developer app + IG professional account per brand.
- **Stripe Billing** — subscriptions (Hustle \$50/mo, Pro \$25/wk, Empire \$50/wk — see section 7), with webhooks updating a `subscriptions` table that gates automation levels.
- **Vercel Cron** — scheduled jobs: nightly demand scoring, post publishing, Monday reports.

Core Supabase tables

```
create table brands (id uuid primary key, owner_id uuid references auth.users,
  name text, ig_account_id text, plan text default 'hustle');

create table products (id uuid primary key, brand_id uuid references brands,
  name text, sku text, stock int, cost numeric, price numeric,
  supplier_email text, lead_time_days int);

create table demand_signals (id bigint generated always as identity primary key,
  brand_id uuid, product_id uuid, source text, -- 'dm' | 'page_view' | 'cart_add' |
  'story_reply'
  created_at timestamptz default now());

create table dm_messages (id bigint generated always as identity primary key,
  brand_id uuid, sender text, body text, is_bot boolean,
  escalated boolean default false, created_at timestamptz default now());

create table content_pool (id uuid primary key, brand_id uuid,
  media_url text, media_type text, caption_draft text, status text default 'pool');

create table scheduled_posts (id uuid primary key, brand_id uuid,
  content_id uuid references content_pool, scheduled_at timestamptz,
  ai_reasoning text, status text default 'awaiting_confirmation');
```

Enable RLS on every table with a policy like

```
brand_id in (select id from brands where owner_id = auth.uid()) .
```

2b. Store Connect — works with any platform brand owners already use

Not every brand owner is on Supabase/Vercel like FLVS — most already have a site on Shopify, WooCommerce, Squarespace, or Wix, or sell purely through IG/WhatsApp DMs with no site at all. flvs.bran has to read stock and orders from wherever they already are, not force a migration. This is done with a **platform adapter layer**: one internal interface (`getProducts()` , `getStock(sku)` , `getOrders(since)` , `updateFulfillment(orderId)`) with a different adapter behind it per platform. Every other engine (Restock AI, chatbot, scheduler, fulfillment) talks to the adapter interface, never to a specific platform's API — so adding a new platform later doesn't touch the rest of the system.

Platform	Integration	Notes
Shopify	Admin API (REST/GraphQL), OAuth app install	Most complete: real-time inventory levels, order webhooks, fulfillment updates. Build as a listed Shopify app for one-click install.
WooCommerce	REST API (API key from WordPress plugin)	Reads stock, order status; can push fulfillment/tracking back via API.
Squarespace	Commerce APIs — Orders API + Inventory API	Available on Core, Plus, Advanced, and Commerce Advanced plans. Orders API supports marking fulfilled + shipment notifications; Inventory API reads/adjusts stock per variant. OAuth or API key auth.
Wix	Wix Stores/Data API + webhooks	Reads catalog and stock; order events via webhook subscription.
Custom-built site (like flvs.life)	Direct Supabase/API integration	Same pattern as section 3b — brand owner's dev team (or you) wires it in directly with an API key.
No website (DM/WhatsApp-only sellers)	flvs.bran is the system of record	Products and stock are entered/managed inside the dashboard itself; no external sync needed.

Onboarding flow: brand owner picks their platform in the dashboard → OAuth authorize (Shopify/Wix/Squarespace) or paste an API key (WooCommerce) → adapter does an initial full sync into the

`products / orders` Supabase tables → a webhook or 5–15 min polling cron keeps it current after that. For anything unsupported, two universal fallbacks: a generic inbound webhook URL, or a scheduled CSV import — both shown in the dashboard's Connect Store tab.

DM-Commerce mode — for “DM to order” brands with no website

A huge share of Caribbean sellers have no site at all: they post the product on IG, caption says “DM to order,” and the whole business runs in the inbox. For them, **flvs.bran itself becomes the store**, and this is a first-class mode, not a fallback:

1. **Catalog lives in the dashboard.** Owner adds products, sizes, prices, and stock directly (phone-friendly form, or snap a photo and let Claude draft the listing). This becomes the system of record the chatbot and Restock AI read from.
2. **The chatbot takes the order.** When a customer DMs “I want di red one in medium,” the bot checks live stock, confirms size/color, collects the delivery address (and drops a map-pin link for lat/lng), and creates a real order record — the same `orders` row a website checkout would create.
3. **Payment without a website.** The bot sends a payment link (WiPay / bank-transfer instructions / card link) or marks the order **cash on delivery** — oDeliver's `payment_on_delivery` flag means the courier collects the cash, which matches how these brands already operate.
4. **Same fulfillment pipeline.** Order confirmed → courier booked → customer gets tracking in the DM thread → Restock AI counts the sale. The seller runs an entire e-commerce operation without ever building a site.
5. **Growth path built in:** when they're ready for a website, their catalog, customers, and order history already live in flvs.bran — one click exports it to a storefront (or you build them a flvs.life-style site as an add-on service).

This mode is also the cheapest to onboard (no platform APIs to connect) and the stickiest — flvs.bran isn't an add-on to their store, it *is* their store.

This means the subscription product works for a brand owner on day one, on whatever they already built, while still being the natural home if they later want a full custom site like FLVS.

3. Restock AI (your priority)

Data it collects

- **Stock:** synced from the brand's connected platform via the Store Connect adapter (section 2b) — Shopify, WooCommerce, Squarespace, Wix, a custom site, or entered directly if they have no website.
- **Sales velocity:** units sold per day over trailing 14/30 days, from the orders table.

- **DM demand:** every incoming DM is scanned by Claude and tagged with the product(s) mentioned → a row in `demand_signals`.
- **Website demand:** a tiny tracking snippet on product pages logs views and cart-adds to `demand_signals`.

The scoring logic (nightly cron)

For each product compute:

```
days_of_stock    = current_stock / avg_daily_sales
demand_trend     = signals_last_7d / signals_prior_7d    (>1.3 = rising)
demand_score     = 0.4*velocity_norm + 0.3*dm_norm + 0.2*cart_norm + 0.1*view_norm
```

Verdicts:

- **REORDER** — `days_of_stock < lead_time_days + safety_buffer` AND `demand_score` above threshold. Suggested qty = `avg_daily_sales × (lead_time + 30) - current_stock`, rounded to supplier minimums.
- **WATCH** — moderate demand, or rising trend but plenty of stock. Re-scored daily.
- **HOLD** — `days_of_stock > 35` or falling trend. Flags cash tied up; suggests a promo instead of a reorder.

Claude writes the plain-language “why” for each verdict so owners trust the call. On the Empire tier, a REORDER verdict auto-drafts a purchase-order email to the supplier with past pricing — owner approves with one tap.

This is deliberately a transparent formula, not a black box: with small-brand data volumes (dozens of orders/week), interpretable heuristics beat ML models and owners can see exactly why the AI said hold.

3b. Order Fulfillment — oDeliver integration (Trinidad)

oDeliver (odeliver.org) has a real REST API with sandbox mode, Bearer-token auth, and WooCommerce/WordPress plugins — so this integration is fully automatable today. Docs: odeliver.org/developers/rest-api. You need a free merchant account, then generate sandbox + production tokens under Settings → API.

The automated flow

```
Customer pays on flvs.life
→ order lands in Supabase `orders` (checkout success handler)
→ Vercel API route /api/fulfillment/dispatch fires:
  1. POST /api/v1/shipments/estimate    (get delivery cost)
```

```

2. POST /api/v1/shipments (book shipment; pickup = your
   point of sale address, dropoff = customer address from checkout)
3. Store returned shipment_id (e.g. "0D-7X9K2M4P") on the order
4. GET /v1/shipments/{shipment_id}/label (print-ready label, PDF)
5. Customer gets a confirmation with tracking — sent via email
   or by the DM chatbot if they ordered through Instagram
→ Cron polls GET /v1/shipments/{id}/track every 30 min
   → status changes (picked up / out for delivery / delivered)
   update the order and message the customer automatically

```

Key API details (verified from their docs):

- **Two service levels:** Standard “Hub & Spoke” (1–2 days, `/api/v1/shipments`) and Instant same-day (`/api/v1/instant-shipments`, supports scheduled slots via `/slots`). Let the customer choose at checkout; the estimate endpoints return live TTD pricing for each.
- **Create shipment** requires pickup + dropoff objects (address, city ID, 11-digit phone, lat/lng), package size array, `payment_on_delivery` flag (driver can collect cash — big for Trinidad, where COD is common), and options like `secure_delivery` (4-digit PIN to complete delivery), `blind_shipping`, and oProtect insurance (3% of declared value, TTD \$101–\$1,500 coverage).
- **Collect lat/lng at checkout** (a map-pin picker on your address form) — instant shipments price by coordinates, and it kills failed-delivery problems with unnumbered Trinidad addresses.
- **Sandbox first:** `odeliver.org/sandbox/v1/...` with sandbox tokens; production tokens on `/api/v1/...`. Mixing them returns 403.
- **Orders table addition:** `shipments (id, order_id, courier text default 'odeliver', shipment_id text, service text, cost numeric, delivery_status int, label_url text)`.

Auto-dispatch is a per-brand toggle: fully automatic, or queue each booking for one-tap owner approval (useful when they need to pack before pickup is booked).

Phase 2 — your own delivery back end

Build the integration behind a **courier adapter interface** from day one: every courier action goes through generic functions (`estimate()`, `createShipment()`, `track()`, `cancel()`, `label()`) with oDeliver as the first adapter. When you launch FLVS’s own fleet, you write one new adapter — the order pipeline, dashboard, and customer notifications don’t change at all.

The in-house system adds three pieces on the same Supabase/Vercel stack: a **driver PWA** (mobile web app: assigned runs, navigation link, proof-of-delivery photo + PIN, COD collection log), a **dispatch engine** (groups paid orders into runs by area, assigns drivers, plans rough route order), and a **branded tracking page** (`flvs.life/track/{id}` — your logo, live status, driver ETA — the brand experience oDeliver can’t give you). Suggested trigger for the switch: when you consistently ship 15–20+ packages/day in a concentrated area, one driver on a daily rate beats ~\$30–40 TTD per delivery — run the math from your own `shipments` cost data, which the system is already collecting. Then keep oDeliver as the fallback adapter for Tobago and off-route areas. Long-term, delivery itself becomes a service you can

sell to the other brands on your platform — your own mini-courier network with flvs.bran as the dispatcher.

4. DM Chatbot with Caribbean dialect

How training actually works

You don't fine-tune a model. You do it with **retrieval + examples**, which is cheaper, instantly updatable, and works per-brand:

1. Owner uploads exported DM threads (IG lets you download your data; also paste-in like the prototype shows).
2. Each conversation is stored in `bot_training`, embedded (Supabase pgvector), and tagged by situation: price question, stock check, shipping, sizing, complaint, greeting.
3. When a customer DM arrives (via Meta webhook → Vercel API route), the system retrieves the 5–8 most similar past conversations and sends them to Claude with a system prompt like: *"You are the voice of [brand]. Reply exactly in the style of these real examples, including the dialect and slang used. Never invent stock or prices — use the product data provided."*
4. Claude's reply is sent back through the Messenger API.

Because the examples ARE the brand's real messages, the bot naturally picks up "wah gwan," "bredrin," "linky," "how much fah" — and each brand's bot sounds like that brand, whether they write Trini, Jamaican patois, Bajan, or standard English. Dialect isn't a setting; it's learned from the uploads.

Guardrails

- Product/stock/price answers are grounded in the live `products` table, never guessed.
 - Confidence scoring: replies below threshold, refund disputes, wholesale/collab inquiries → escalate to the owner (push notification + flagged in dashboard) instead of answering.
 - Every bot reply is logged so owners can correct bad ones — corrections go straight back into the training set.
 - Every DM also feeds `demand_signals` for Restock AI. The two systems compound.
-

5. AI Post Scheduler

1. Owner uploads content into the **pool** (Supabase Storage) — photos, reels, stories, with or without captions.
2. A weekly cron pulls the brand's **IG Insights**: follower activity by hour/day, reach by media type, engagement history per post.

3. Claude assigns each pool item a slot: format-to-day matching (e.g., reels on peak evenings, UGC on weekend mornings), spacing rules, drop-hype timing near restocks. It drafts a caption in the brand voice (reusing the chatbot's dialect examples).
4. Owner sees the proposed week and taps **Confirm** or **Reslot** per post — exactly the flow in the prototype.
5. Confirmed posts auto-publish at time via the IG Content Publishing API (Vercel cron checks `scheduled_posts` every 5 minutes).

6. Back-end automations — what they are and which tier gets them

The automations are gated by one rule that also explains itself to customers: **alerts at Hustle, actions at Pro, campaigns at Empire**. The base tier's automations *tell you about money* (cheap to run, build daily habit, and constantly advertise what the higher tiers would do about it). Pro's automations *recover money* — the concrete reason to upgrade. Empire's automations *grow money on their own* — ongoing campaigns that compound, fitting the “runs itself” promise. Gating everything at the top tier was considered and rejected: it would hollow out the lower tiers and slow adoption; alert-tier automations at the base are the best upgrade advertising the product has.

Automation	Type	Tier	What it does
Weekly Boss Report	Alert	Hustle	Monday summary: sales, reorders, best post, DMs handled. Cheap to build, massive perceived value.
Drop Hype Detector	Alert	Hustle	DM/story-reply spikes before a drop → alert to raise qty or open pre-orders. You act on it.
Abandoned Cart Rescue	Action	Pro	Brand-voice DM/email to cart abandoners; recovers 8–15% of lost checkouts automatically.
Supplier Auto-PO	Action	Pro	REORDER verdicts auto-draft the purchase order for one-tap approval.
Price & Promo Advisor	Action	Pro	Pairs with HOLD verdicts; builds the bundle/flash-sale most likely to clear slow stock.
	Campaign	Empire	

Automation	Type	Tier	What it does
Customer Segments & Win-back			Auto-segments customers; runs win-back campaigns after 60 days unattended.
Review & UGC Collector	Campaign	Empire	Post-delivery review/ photo ask with discount code; best UGC feeds the content pool — compounding loop.
Fraud & Chargeback Shield	Campaign	Empire	Screens every order for risk signals before shipping.

Downgrades auto-disable automations above the new tier. Locked automations stay visible in the dashboard with a lock badge — seeing them is part of the ladder.

Later additions (slot into the same rule): multi-currency + island shipping-rate helper (TT/JA/BB rates and customs notes are a real regional pain point competitors ignore), and a WhatsApp Business channel for the chatbot — WhatsApp is huge in the Caribbean and would be a serious differentiator.

7. Business model — a growth ladder priced around the TT\$ card reality

Two rules shape the pricing. **Rule 1 (payment reality)**: most TT debit cards cap international/online spending at roughly US\$50 per day, so no single charge ever exceeds US\$50 — base bills monthly at exactly \$50, higher tiers bill weekly. **Rule 2 (ladder psychology)**: every tier includes *every feature* — what changes is **how much automation you get before the system asks you to step in**. The base tier is a working taste of everything with caps a growing brand will genuinely hit; the top tier is full autopilot. Upgrading isn't buying missing features, it's buying back your own time — which makes the expensive package the obvious aspiration rather than a luxury.

HUSTLE — US\$50/month (one charge, exactly at the card limit). *“The system works, you still press the buttons.”* Full DM-commerce mode (it's how no-website sellers operate, so it can't be gated), chatbot with dialect training capped at **300 replies/mo**, Restock verdicts (REORDER/HOLD/WATCH) but **you write your own supplier orders, 10 AI-scheduled posts/mo**, courier booking at **one tap per order** (not auto), 1 IG account. The caps are calibrated so a brand doing a few orders a day outgrows them in weeks, not months.

PRO — US\$25/week (≈\$100/mo). *“The system runs, you approve.”* Unlimited chatbot replies, unlimited scheduled posts, Restock AI drafts the supplier PO for one-tap approval, auto-dispatch to courier,

abandoned-cart rescue, drop-hype alerts, weekly Boss Report. This is the tier the product is designed around.

EMPIRE — US\$50/week (≈\$200/mo). *“The brand runs itself, you check the report.”* Everything fully automatic — supplier ordering, dispatch, win-back campaigns, customer segments — plus multi-brand/multi-account, priority support, and onboarding. Positioned openly as the goal: the dashboard shows every owner, whatever their tier, an “on Empire this would have been done for you” line item so the aspiration is always visible.

The upgrade engine (build this, it matters more than the price points): the dashboard actively shows money left on the table at the current tier — “Your bot went quiet on 41 customers after hitting your reply cap (est. TT\$2,400 in missed orders)” or “3 REORDER verdicts sat 5 days waiting for you to write POs; Pro would have drafted them same-day.” Every nudge is a real number from their own data, not marketing copy. That converts far better than feature comparison tables, and it’s honest — the system genuinely is leaving value un-captured at lower tiers.

Billing mechanics: weekly billing is native in Stripe (`interval: 'week'`). Enable **smart retries** — a \$25–50 charge that fails because the day’s limit was already spent usually clears next day; most “failed payments” in this market are limit collisions, not cancellations, so retry before dunning. Annual prepay later only via methods that bypass card limits (bank transfer/WiPay). 14-day free trial **of Pro, not Hustle** — let them feel the automated version, then the caps of Hustle create the pull back up. Feature-gating read from the `subscriptions` table. Your own FLVS results are the marketing: “the system that runs FLVS.”

Weekly billing also smooths your cash flow and matches how brand money actually arrives — owners feel \$25/week from weekend sales far less than a surprise \$100 monthly hit on a card that would decline anyway.

8. Adding it to www.flvs.life (Supabase + Vercel)

Since flvs.life is already Supabase + Vercel, flvs.bran isn’t a separate product you bolt on — it lives in the same project:

Step 1 — Same repo, gated route. Add the dashboard at `flvs.life/ai` (or subdomain `app.flvs.life` via Vercel domains) as a route group in your Next.js app, protected by Supabase Auth. Your public store and the SaaS share one codebase and one database.

Step 2 — Create the tables from section 2 in your existing Supabase project, with RLS on.

Step 3 — Instrument your store. In your product page components, log `page_view` and `cart_add` events to `demand_signals` (one `supabase.from('demand_signals').insert(...)` call). Point your order/checkout success handler at the `orders` table. Restock AI now has live FLVS data on day one.

Step 4 — Meta app. Create a Meta developer app, add Instagram Messaging + Content Publishing + Insights permissions, set the webhook URL to `flvs.life/api/webhooks/instagram` (a Vercel API route). Connect your FLVS IG professional account first; brand-owner customers OAuth their own accounts later. (Meta app review is the slowest step — start it early.)

Step 5 — Claude API. Add `ANTHROPIC_API_KEY` to Vercel env vars. Three API routes: `/api/bot/reply` (webhook-triggered DM replies), `/api/restock/score` (cron), `/api/scheduler/plan` (weekly cron). Cron config in `vercel.json`:

```
{ "crons": [
  { "path": "/api/restock/score", "schedule": "0 6 * * *" },
  { "path": "/api/scheduler/publish", "schedule": "*/5 * * * *" },
  { "path": "/api/reports/weekly", "schedule": "0 12 * * 1" }
]}
```

Step 6 — Site chat widget (optional but easy). Drop a chat bubble component on `flvs.life` that calls the same `/api/bot/reply` route — the DM bot doubles as your website live chat.

Step 7 — Stripe subscriptions + webhook → `subscriptions` table when you open it to other brands.

Build order

1. **Weeks 1–2:** tables + store instrumentation + Restock AI cron + dashboard restock tab (FLVS only). *You get value immediately.*
2. **Weeks 2–3:** oDeliver merchant account + sandbox integration → auto-dispatch pipeline live (section 3b). Small, high-payoff build since their API is straightforward.
3. **Weeks 3–4:** Meta app + webhook + chatbot with your DM training data; website chat widget.
4. **Weeks 5–6:** content pool + scheduler + IG publishing.
5. **Weeks 7–8:** Stripe, multi-tenant onboarding, weekly report → open to first paying brands.

Running costs (early stage)

Vercel Pro ~\$20/mo, Supabase Pro ~\$25/mo, Claude API roughly \$5–30/mo per active brand depending on DM volume (Haiku-class models keep chatbot replies cheap), Stripe 2.9% + 30¢ per charge (note: weekly billing means 4× the fixed 30¢ fees vs monthly — about \$1.20/mo extra per Pro/Empire subscriber, negligible against the reliability gain). Even a \$50 Starter subscriber is solidly profitable.

9. Beyond clothing — other online business types this fits

Nothing in the core engines is apparel-specific — Restock AI is a stock + demand-signal formula, the chatbot is trained on whatever conversations you feed it, the scheduler works off any brand's IG data, and Store Connect (section 2b) already reads from generic e-commerce platforms. That means once the

clothing vertical is proven with FLVS, the same product opens to other Caribbean online sellers with only light, mostly copy/config changes — not a rebuild.

Strong fits, minimal changes needed - Beauty & cosmetics brands — identical model to apparel: SKU-based stock, drop culture, heavy DM/IG selling. Restock AI and the scheduler need zero changes. - **Food & beverage / home-based kitchens (bake shops, meal prep, sauces/pepper products)** — Restock AI adapts to ingredient/batch stock instead of finished SKUs; the courier integration (section 3b) is even more central since most of these are 100% delivery-driven. - **Jewelry & accessories, phone/electronics accessories resellers** — same low-SKU-count, high-DM-volume pattern as streetwear. - **General small e-commerce (any Shopify/WooCommerce store)** — Store Connect already targets this; no product change required, just onboarding.

Good fits, need a bolt-on module - Service-based businesses (salons, detailing, photographers, event rentals) — Restock AI isn't relevant, but the chatbot (booking questions, availability, pricing) and scheduler (posting availability/portfolio content) still apply directly. Would need a booking-calendar module instead of a stock table — same architecture, new adapter. - **Multi-vendor markets / pop-up collectives** — the multi-brand support already planned for the Empire tier extends naturally to an “umbrella” account managing several small vendors under one dashboard.

Positioning implication: keep “flvs.bran” as the product name tied to your story, but consider the pitch as “AI back-office for Caribbean online sellers” rather than “clothing brand tool” once you're past the first cohort — it doesn't cost you anything in the build, and it roughly triples the addressable market in Trinidad alone.

10. Onboarding, login portal & data-access terms

Branding: the flvs. wordmark (heavy italic sans with the “co” roundel) appears on every surface — login, wizard, dashboard header, reports, emails. The admin login portal uses the brand red (#CB352E, sampled from the fan) as a full-bleed background with the white wordmark, matching the “BUY HER FLVS.” merch look. Product screens stay clean off-white with red accents.

Subscribe flow (4 steps, built in the prototype):

1. **Brand info** — brand name, owner, email, 11-digit phone (oDeliver requires it), point-of-sale/pickup address, and business vertical. This one form powers courier pickups, bot sign-off, and the dashboard.
2. **Pick a rung** — Hustle / Pro / Empire with the tier taglines; every signup gets a 14-day Pro trial regardless of choice.
3. **Integrations** — platform selector (DM-Commerce needs no keys; others take an API key/token), social connects (Instagram required — the bot and scheduler run on it; WhatsApp and TikTok optional), and an optional oDeliver merchant token for auto courier booking. Everything skippable except IG, all editable later in Settings.

4. **Terms & data access** — scrollable agreement with a required checkbox; acceptance timestamped on the account record.

The data-access terms (the honest version, written to build trust rather than hide):

- *What the system accesses and why*: continuous read access to stock, orders, sales, site events, IG DMs and insights — the automations cannot run without it.
- *What staff can see*: business values are not viewed by personnel in day-to-day operation — the system processes them automatically. But the terms state plainly that flvs.bran personnel **do have the technical ability to access dashboard data**, and will use it only to operate the service, investigate faults, or support the owner at their request — with support access logged and visible to the owner in Settings. (An access log is cheap to build on Supabase — one table — and turns a scary clause into a trust feature.)
- *What we never do*: no selling or sharing data with third parties, advertisers, or other brands on the platform; per-brand isolation enforced by Row Level Security.
- *Owner control*: disconnect integrations, export data, or cancel any time; deletion within 30 days of cancellation.
- *Billing and bot-liability clauses* round it out: the US\$50 charge ceiling, weekly billing, retry-before-interruption, and the owner's responsibility for chatbot training content with automatic escalation of refunds/disputes.

Legal note: have a T&T attorney review the final terms and data-protection wording before taking paid signups — this blueprint's version is a product spec, not legal advice.

11. Domain architecture — where everything lives

flvs.life	→ FLVS the swimwear brand (your store, customer #1)
flvs.life/bran	→ bran landing page (header link on the main site)
app.flvs.life	→ the backoffice – brand owners log in here
admin.flvs.life	→ platform admin portal (staff only)
shop.flvs.life/{handle}	→ each subscriber brand's storefront
shop.flvs.life/{handle}/track/{code}	→ that brand's customer tracking page
{brand's own domain} (optional)	→ CNAME-mapped to the same storefront

Subscriber stores deliberately do **not** live under flvs.life paths. Another brand's customers shouldn't shop inside a swimwear brand's website — it dilutes both identities — and subdomain separation keeps each brand's cookies, checkout, and SEO independent. It also makes graduation painless: when a brand buys their own domain, it maps onto the same store via Vercel custom domains with zero migration.

Implementation is still one codebase: a single Next.js app serves `shop.flvs.life/[handle]`; middleware resolves the handle against the `brands` table in Supabase and loads that brand's catalog, colors, pickup point, and bot voice — the exact pattern the storefront demo simulates with its `?brand=` parameter. A new subscriber's store therefore exists the moment they finish the signup wizard.

Prototype: `flvs-brandai-prototype.html` — a clickable demo of the dashboard described here.